PENTEST-STUDIO · TECHNICAL WRITEUP

HACK THE BOX · LINUX

Codify

Complete technical report · reproducible exploitation

Codify machine solved

HTB Codify · May 06, 2026

Machine	**Codify**
Platform	**Hack The Box**
Status	**Retired**
Difficulty	**Easy**
System	**Linux**
Date	**May 06, 2026**
Author	**Ramon Santos Sabarich / r4ms4nt**

Standardized technical report for archiving, reference, and traceability.

READINGGUIDE

Contents

Didactic technical report for Codify. The structure preserves the real laboratory sequence: initial enumeration, dominant surface identification, exploitation, lateral movement, privilege escalation, and technical closure.

Hack The Box - Codify

Didactic technical report

Machine: Codify
Platform: Hack The Box
System: Linux
Difficulty: Easy
Work date: 2026-05-06, Europe/Madrid time zone
Status: Solved
Initial user obtained: `svc`
Lateral user: `joshua`
Final user: `root`

1. Technical synopsis

Codify exposes a web application developed with Node.js/Express that allows users to run JavaScript snippets inside a sandboxed environment. The application itself documents the use of `vm2` and links to version `3.9.16`, which turns the web surface into a candidate path for a sandbox escape.

The resolution relies on a progressive chain of evidence:

1. initial enumeration with `Inici-HTB`;
2. identification of a Node.js/Express application on ports `80` and `3000`;
3. detection of `vm2 3.9.16` as the sandboxing technology;
4. benign validation of the `/run` endpoint;
5. confirmation that direct use of sensitive modules such as `child_process` is blocked;
6. exploitation of a public vulnerability compatible with the observed version;
7. shell obtained as `svc`;
8. search for local application artifacts;
9. extraction of a bcrypt hash for `joshua` from a SQLite database;
10. offline cracking of the hash and SSH access as `joshua`;
11. review of `sudo` permissions;
12. analysis of a backup script executable as `root`;
13. abuse of an unsafe Bash comparison;
14. capture of the real password by observing processes with `pspy`;
15. authentication as `root` and reading of the final flag.

The complete chain can be summarized as follows:

2. Lab preparation and startup

2.1 Use of Inici-HTB

The case started by running the custom `Inici-HTB` script, used to prepare the working environment, check connectivity, and generate an initial evidence baseline.

This script does not exploit anything. Its value is to keep the lab organized from the beginning and leave a minimal working structure:

The didactic idea behind this phase is important: before looking for vulnerabilities, it is necessary to know what real attack surface exists. In a simple Linux machine, a poor start usually causes two mistakes: jumping into exploitation without evidence, or wasting time on secondary branches.

2.2 Initial evidence

The machine responded to ping:

The `63` TTL and the output from `whichSystem.py` pointed to Linux:

The full port scan detected three open ports:

The service scan refined the surface:

Additionally, port `80` showed a redirect to:

2.3 Reading phase 1

The first methodological decision was clear:

SSH was noted as a secondary path because it was open, but no credentials existed yet. The dominant surface was web, with two strong signals:

3. Hostname normalization and HTTP surface comparison

3.1 Resolving codify.htb

Nmap had already revealed the hostname `codify.htb`, so it was added to `/etc/hosts`:

Check:

Output:

This step was necessary because port `80` redirected to the hostname. If the attacking machine did not resolve `codify.htb`, HTTP requests would not accurately represent the application's real behavior.

3.2 Comparing IP, hostname, and port 3000

HTTP headers were reviewed by IP, by hostname, and through the direct `3000` port:

The IP redirected to the hostname:

The hostname returned `200 OK` and showed Express behind Apache:

Port `3000` returned the same length and also included `X-Powered-By: Express`:

The interpretation was that Apache on port `80` acted as a frontend or proxy, while port `3000` exposed the Express application directly. In both cases, the same backend was reached.

3.3 Extracting public routes

Visible resources were extracted from the main page:

Observed routes:

`robots.txt` did not exist as a valid resource:

`whatweb` confirmed the technology reading:

3.4 Closing initial WEB-BASE

The application did not expose a CMS, a login, or an authenticated panel. No public downloads or artifacts appeared either. The relevant signal was different: the application presented itself as a platform for running Node.js code inside a sandbox.

The visible text stated:

That changed the focus. The web surface was no longer generic; it was an application that executes untrusted code.

4. Identifying the sandbox technology

4.1 Downloading informational pages

The relevant public routes were saved:

The reason for saving these pages was to preserve local evidence and search technical terms without depending on the browser.

4.2 Searching for key terms

Indicators related to code execution, sandboxing, and Node.js modules were searched:

The `/about` page identified the sandbox technology:

It also linked to:

This was a very strong version signal, although it must be described precisely: it is a version observed from the application, not a version confirmed from the filesystem.

4.3 Restrictions declared by the application

The `/limitations` page stated that certain modules were restricted:

It also listed allowed modules:

This information helps explain the application's defensive model. This is not uncontrolled Node.js execution; there is an explicit attempt to block dangerous modules. However, that layer does not prove the sandbox is secure.

4.4 Real editor flow

The `editor.html` file revealed how code is sent to the backend:

The browser encodes the content with `btoa()`:

and sends it to:

The dominant technical surface becomes:

5. Benign validation of the /run endpoint

5.1 Why validate before exploiting

Before using a public PoC, the flow was validated benignly. This prevents mistaking formatting, Base64, JSON, or endpoint problems for an exploitation failure.

A harmless code snippet was prepared:

and encoded in Base64:

Output:

5.2 Testing /run

The browser behavior was reproduced from the terminal:

Response:

The same test worked against the direct `3000` port:

Response:

The interpretation is clear: the `/run` endpoint accepts JSON, expects `code` in Base64, executes JavaScript inside the backend, and returns the output in the `output` field.

5.3 Restricted module validation

The next check was whether the restriction on `child_process` was real. The equivalent of this was sent:

encoded as:

Request:

Response:

This result does not discard the exploitation path. On the contrary, it clarifies the defensive model: the application blocks direct imports of dangerous modules and delegates real isolation security to `vm2`.

The technical question is no longer:

The question becomes:

6. Public candidate: sandbox escape in vm2

The review of public vulnerabilities associated with `vm2 3.9.16` leads to CVE-2023-30547, a sandbox escape vulnerability in `vm2`. According to the working notes, this candidate affects versions up to `3.9.16`, is related to improper sanitization of exceptions, and was later fixed.

At this point, three levels are separated:

Verified fact

The application uses `vm2` and links to `3.9.16`.

Reasonable inference

The version in use may be vulnerable to CVE-2023-30547.

Pending until validation

The vulnerability must be tested against the real `/run` endpoint and demonstrate execution outside the sandbox.

7. Validating the sandbox escape

7.1 Minimal test with id

The initial validation was performed from the page:

A public PoC adapted to execute `id` was pasted:

The output was:

The importance of this test is that `child_process` was blocked through direct import. Getting the output of `id` proves that execution is no longer limited to the normal sandbox flow and that the context of the backend Node.js process has been reached.

7.2 Preparing the reverse shell

A local `rev.sh` script was created on the attacking machine:

Content:

An HTTP server was started to serve the script:

And a listener was opened to receive the connection:

Before executing the PoC, the real VPN interface IP had to be confirmed:

Relevant output:

This control avoided mixing IP addresses in the script and in the PoC `curl` call.

7.3 PoC to obtain a shell

The PoC was modified in `/editor` to download and execute `rev.sh`:

The listener received a connection from the victim:

The shell was minimally stabilized:

Context validation:

Output:

The initial access phase is closed: the initial user is `svc`.

8. Local enumeration after foothold

8.1 Identifying interactive users

`/etc/passwd` was reviewed to identify local users with a shell:

Relevant users:

This output guided lateral movement: if the application stored credentials or hashes, `joshua` was the candidate local user.

8.2 Searching for web artifacts

The reference log pointed to `/var/www/contacts`, but that path did not exist on the worked instance. Instead of insisting on an unverified route, `/var/www` was enumerated:

Relevant output:

The correct path was:

This adjustment is methodologically important: the reference guides, but the machine evidence rules.

8.3 Confirming the SQLite database

The real directory was reviewed:

Output:

The machine had `sqlite3` installed, so transferring the database was not necessary for reading it.

9. Extracting a hash from tickets.db

9.1 Enumerating tables

The tables were listed:

Output:

The `users` table became the priority because it could contain credentials or reusable hashes.

9.2 Dumping the users table

Schema and content were displayed:

Output:

Content:

The `$2a$12$` prefix classified the hash as bcrypt.

9.3 Offline cracking

The hash was saved on the attacking machine:

Hashcat was launched in bcrypt mode:

Result:

Credential obtained:

The password was cracked offline, not on the victim. This reduces noise on the target system and keeps the credential phase separated from local enumeration.

10. Lateral movement through SSH

From phase 1, SSH was known to be open:

The recovered password was tested against the local user `joshua`:

Password:

Access confirmed:

The user flag was read:

Output:

The lateral movement chain is closed:

11. Sudo enumeration as joshua

11.1 Reviewing sudo permissions

The first local privilege escalation check was:

Password used:

Relevant output:

The priority changed: the SQLite database and web application were no longer important. The main path was now a custom script executable as `root`.

11.2 Inspecting the script before running it

Permissions, file type, and content were reviewed:

Permissions:

Type:

Relevant content:

It then uses the real password in MySQL commands:

and:

12. Vulnerability analysis of the script

The script contains two useful weaknesses in sequence.

12.1 Unsafe Bash comparison

The critical line is:

In Bash, inside `[[ ... == ... ]]`, the right side of the comparison may behave as a pattern. If `*` is entered, it can match any password.

This was validated by running:

When the script asked:

the following was entered:

Output:

The bypass was confirmed.

12.2 Password exposure through process arguments

After the bypass, the script executes `mysql` and `mysqldump` with the password on the command line:

This even generates warnings:

The warning is an excellent clue: if the password travels as a process argument, it can be observed temporarily with a tool such as `pspy`.

13. Capturing the real password with pspy

13.1 Preparation on the attacking machine

`pspy64s` was downloaded:

13.2 Transfer to the victim

From the SSH session as `joshua`:

`pspy` was left running in one SSH session.

13.3 Second SSH session to trigger the script

Another SSH session as `joshua` was opened and the following was executed:

It is important to distinguish the two password requests:

Here goes `joshua`'s password:

Then the script request appears:

Here goes the wildcard:

13.4 Captured password

`pspy` captured the `mysqldump` process:

Recovered password:

The line confirms three facts:

1. the process runs as `UID=0`;
2. the script uses the real password read from `/root/.creds`;
3. that password is exposed in the process arguments.

14. Final escalation to root

Password reuse was tested with `su root`:

Password:

Validation:

Output:

The root flag was:

15. Final technical summary

15.1 Exploitation chain

15.2 Flags

15.3 Credentials and secrets from the case

16. Reusable lessons

16.1 Read the application before searching for exploits

Codify shows that a simple website can reveal its own attack path if its pages are read carefully. `/about`, `/limitations`, and `/editor` provided more value than early generic fuzzing.

Reusable pattern:

16.2 Distinguish direct blocking from real security

Blocking `child_process` did not make the application secure. It only blocked a direct path. Real protection depended on `vm2`, and the observed version was vulnerable.

Lesson:

16.3 Validate the flow before exploiting

Before using the PoC, `/run` was confirmed to accept JSON, require Base64, and return output in `output`. This validation avoided interpretation errors.

Lesson:

16.4 Real evidence corrects the writeup route

The expected path was `/var/www/contacts`, but the machine had `/var/www/contact`.

Lesson:

16.5 Local application databases are frequent pivots

`tickets.db` contained `joshua`'s hash. This pattern is very common after a web foothold: the application often has local databases, `.env` files, configurations, or artifacts that enable movement to another user.

16.6 Cracking is performed offline

The bcrypt hash was extracted and cracked on the attacking machine. No cracking was run on the victim.

Lesson:

16.7 Do not run sudo scripts without reading them

`sudo -l` showed a custom script. The correct path was to read it before running it. The bug was in its logic: reading `/root/.creds`, unsafe comparison, and use of the password as an argument.

16.8 Warnings matter

The message:

was not noise. It was a direct clue that the password could be seen in processes.

16.9 Distinguishing password requests avoids errors

During escalation there were two different password requests:

Confusing them causes authentication failures unrelated to the vulnerability.

16.10 Operational incidents that are not machine findings

During the lab, local incidents appeared that are useful to remember but must not be confused with Codify vulnerabilities:

- using `path` as a variable in `zsh` temporarily broke command resolution for tools like `curl`, `grep`, and `sort`;
- a `heredoc` with internal Markdown blocks can be copied incorrectly;
- dirty shell pasting was corrected with:

These incidents are useful for operational learning, but they are not part of the machine exploitation.